

बोला मराठी (Bola Marathi) — Comprehensive Development Guide

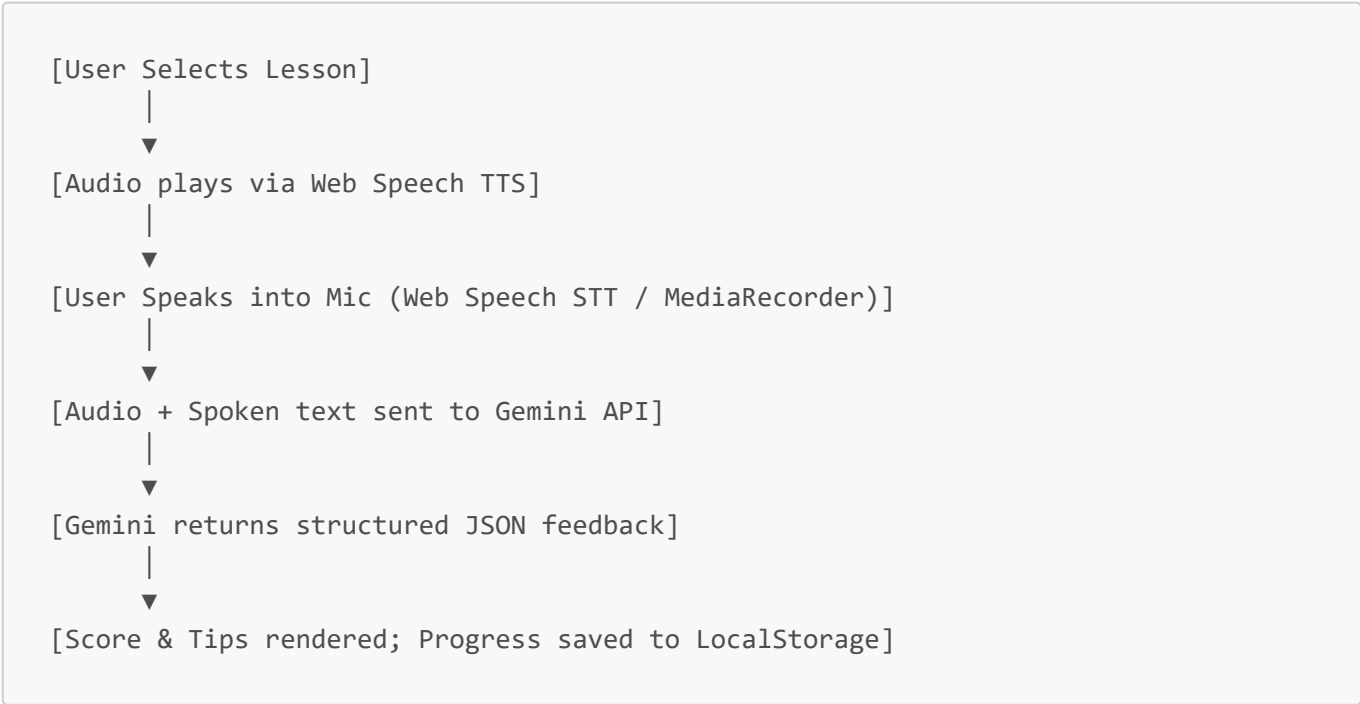
Welcome to the official development guide for **बोला मराठी**! This document explains the architecture, features, tools, and technical implementation details used to build this Progressive Web App (PWA).

1. High-Level Architecture

The application is built as a **Single Page Application (SPA)** using native, vanilla web technologies. It has no build step, no npm dependencies, and runs entirely in the client browser.

```
c:\Translation\  
├─ index.html           # Single-page application shell  
├─ manifest.json        # PWA configuration  
├─ sw.js                # Service Worker (handles offline caching)  
├─ css/  
│   └─ styles.css       # Design system & styles  
├─ data/  
│   └─ lessons.json     # Compiled 12-module curriculum database  
└─ js/  
    ├─ app.js           # Main application controller & router  
    ├─ ui.js            # UI renderer, screen builder, and animations  
    ├─ curriculum.js    # Curriculum data model & loader  
    ├─ audio.js         # TTS playback and audio recording (STT)  
    ├─ ai-feedback.js   # Gemini API feedback assessor  
    └─ progress.js      # LocalStorage stats & spaced repetition tracker
```

Flow of the AI Learning Loop:



2. Core Modules & Technical Details

A. The Front-End Shell & PWA (`index.html`, `manifest.json`, `sw.js`)

- **SPA Structure:** `index.html` serves as a container. All views (Onboarding, Dashboard, Lesson Player, Review Mode, Settings) are generated dynamically using Javascript DOM manipulation inside `js/ui.js`.
- **Installability:** `manifest.json` defines the display mode (`standalone`), colors, name, and app icons. This is what allows Chrome/Edge to display the "Install" button.
- **Offline Caching:** `sw.js` registers a service worker. It caches the HTML shell, stylesheets, javascript logic, and `lessons.json` using a **cache-first strategy**. When offline, the app still loads, displays modules, and plays audio. Only the Gemini AI feedback requires internet connection.

B. Text-to-Speech (TTS) Engine (`js/audio.js`)

- **Technology:** Uses `window.speechSynthesis` (Web Speech API).
- **Marathi Voice:** Filters system voices to locate `mr-IN` (Marathi - India).
- **Controls:** Offers speed options:
 - **Slow:** `0.6x` rate (helps beginners catch vowel differences)
 - **Normal:** `1.0x` rate
 - **Fast:** `1.2x` rate

C. Speech-to-Text & Recording (`js/audio.js`)

- **Chrome/Android:** Uses the browser's native `webkitSpeechRecognition` configured for `mr-IN`. This converts the spoken audio to Marathi Devanagari text locally.
- **Safari/iOS & Fallback:** Since iOS browsers have limited support for Marathi recognition in the browser, the app falls back to the **MediaRecorder API** to record raw `.webm/.wav` audio. This raw audio is sent as a Base64 string directly to the Gemini API for transcription.
- **Waveform Visualization:** Uses the browser's `AudioContext` and `AnalyserNode` to draw a real-time pulsing waveform on the screen while the microphone is active.

D. Gemini AI Feedback Engine (`js/ai-feedback.js`)

- **Model:** `gemini-2.5-flash` (chosen for high speed, multilingual capability, and structured JSON output).
- **How it Assesses:**
 1. Sends the expected phrase, the user's transcription (or raw audio), and the lesson context.
 2. Prompts Gemini using **Structured Output** (`responseSchema` of JSON type) to return:
 - `score`: A numeric score (0 to 100).
 - `accuracy`: A grade (excellent, good, needs improvement).
 - `feedback`: Sincere, friendly text in English.
 - `word_scores`: A breakdown of scores and pronunciation tips for individual words.
 - `encouragement`: A motivational emoji-rich phrase.

E. Progress Tracking (`js/progress.js`)

- **LocalStorage:** Saves user data without needing a back-end database.
- **Metrics Tracked:**
 - **XP (Experience Points):** Earned based on score tiers (e.g., +15 XP for score ≥ 90%).
 - **Streak:** Counts consecutive days practiced.
 - **Spaced Repetition:** Words scored below 70% are marked for review. They appear in "Review Mode" automatically.

3. The 60-Hour Curriculum Database ([data/lessons.json](#))

The curriculum consists of **12 Modules, 100 Lessons, and 835 phrases**. It is structured to guide the user from single words to advanced cultural idioms:

Module	Title	Lessons	Difficulty	Focus
M1	Greetings & Basics	15	Beginner	Hello, please, thank you, sorry, basics
M2	Introducing Yourself	15	Beginner	Name, profession, family, age, future plans
M3	Numbers & Time	15	Beginner	Counting, telling time, days, months, seasons
M4	At the Market	15	Beginner	Groceries, buying, weights, bargaining, paying
M5	Food & Dining	5	Intermediate	Ordering, taste profiles, requests, kitchen
M6	Directions & Travel	5	Intermediate	Maps, left/right, tickets, train/bus schedules
M7	Daily Routine	5	Intermediate	Chores, morning schedules, office routines
M8	People & Descriptions	5	Intermediate	Hair, height, personality, emotions, family
M9	Health & Emergencies	5	Intermediate	Sickness, doctor talk, pharmacy, calling police
M10	Social Conversations	5	Intermediate	Weather, hobbies, inviting friends, small talk
M11	Work & Business	5	Intermediate	Office tasks, professional intro, phone calls
M12	Culture & Celebration	5	Intermediate	Festivals (Ganeshotsav), blessings, idioms

4. Tools, APIs, & Keys Used

1. **Google AI Studio:** Where you generate your Gemini API Key.
 - URL: aistudio.google.com
2. **Gemini API:**
 - Endpoint: https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=YOUR_API_KEY

3. **Web Audio API:** `AudioContext`, `AnalyserNode`, and `MediaStreamAudioSourceNode` used for animating the microphone waveform.
4. **Service Worker API:** Used in browser background to intercept fetch requests and serve static resources locally.
5. **Python HTTP Server:** A light, built-in tool to run a server locally:

```
python -m http.server 8000
```

5. How to Save/Export this Guide as a PDF

Since this is a markdown file, you can convert it to a beautiful, single PDF document:

1. In VS Code (Recommended):

- Install the extension **Markdown PDF** (by yzane).
- Open this file (`bola_marathi_guide.md`).
- Press **F1** or **Ctrl + Shift + P**, search for **Markdown PDF: Export (pdf)**, and click it.
- A PDF file will be generated in the same directory!

2. In Chrome / Edge:

- Open the app locally (`http://localhost:8000/bola_marathi_guide.md` if loaded, or use a markdown viewer extension).
- Right-click anywhere -> select **Print...** -> Choose **"Save as PDF"** as the destination -> Click **Save**.